

Grundlagen digitaler Systeme WS 25/26

Hausaufgabe 5

Abgabe: 22.01.2026, 23:59

Testate: 26.01.2024–04.02.2024

Bitte geben Sie in den folgenden Aufgaben ihre Lösungsschritte stets explizit an, falls nicht anders angegeben. Nicht kommentierte Lösungen werden **nicht** gewertet.

Aufgabe 1 *Metriken*

In der Moodle-Aufgabe finden Sie ein Programm `primesum.c`. In dieser Aufgabe sollen Sie die Effizienz verschiedener Variationen dieses Programms anhand von Metriken evaluieren.

1. Zeilen 6/7 und 8/9 sind je einander ausschließend. Außerdem unterstützen die meisten C-Compiler die Optionen `-O0` und `-O3`. Zeile 7 ist nicht Standardkonform, funktioniert jedoch trotzdem mit `gcc` und `clang`.

Option	Erste Variante	Zweite Variante
Ganzzahlbreite	Zeile 6 (64-bit)	Zeile 7 (128-bit)
Modulo	Zeile 8 ($1e9 + 7$)	Zeile 9 (16^8)
Compileroptimierung	<code>-O0</code>	<code>-O3</code>

Als Basisprogramm nehmen wir für alle drei Optionen die erste Variante. Suchen Sie sich 4 weitere Variationen anhand dieser möglichen Kombinationen aus. Überlegen Sie sich, wie sich die Ausführungszeit jeder dieser Varianten gegenüber der des Basisprogrammes (überall die erste Variante) verhält. Nachdem Sie ihre Hypothesen festgehalten haben, testen Sie diese, indem Sie die Ausführungszeit jeder Ihrer Variationen und des Basisprogramms messen. Für einfaches Benchmarking ist [hyperfine](#) zu empfehlen. Vergleichen Sie die Messergebnisse mit ihren Erwartungen. Gibt es Unterschiede?

2. Energieeffizienz ist auch in der heutigen Zeit relevant. Wählen Sie eine Variation aus. Messen Sie die Ausführungszeit dieser auf 2 verschiedenen Hardwaregeräten. Messen Sie gleichzeitig den „Energieverbrauch“ dieser Geräte. Nutzen Sie dafür beispielsweise die Herstellerspezifikationen der CPU.

Berechnen Sie den Energieverbrauch pro Ausführung des Programms für jedes der Geräte. Berechnen Sie anhand des [Handelspreises von Elektrizität in Deutschland für die aktuelle Viertelstunde während der Vortagsauktion](#) die Elektrizitätskosten für die Ausführung des Programmes. Was sind Faktoren, die dieses Ergebnis verfälschen?

Aufgabe 2 Cache Adressen und Speicherzugriffe

Wir betrachten einen direkt abbildenden Cache in einem System mit einem 8 Bit breiten Adressbus. An jeder Speicheradresse wird ein halfword (2 Bytes) abgespeichert. Die Adresse ist wie folgt unterteilt:

- Bit 5-7: Cache-Tag
- Bit 2-4: Cache-Block
- Bit 0-1: Offset

Bearbeiten Sie die folgenden Aufgaben.

1. Wie groß ist ein Block des Caches? Wie groß kann der Cachespeicher maximal sein, in Bytes und in KiBiBytes?
2. Der Cache soll weiterhin die selbe Größe besitzen, aber die Blockgröße verändert sich auf ein Word. Wie kann sich der Aufbau der Adresse verändern? Wie verändert sie sich, wenn der Cache zusätzlich auch noch zweifach Assoziativ ist? Gehe dabei weiterhin von einer 8-Bit Adresse aus und gebe zusätzlich an, ob man so mehr Speicherblöcke adressieren kann.
3. Wir gehen wieder von unserem ursprünglich gegebenem Cache aus. Wir führen eine Reihe an Lesebefehlen aus. (Zur Vereinfachung ist das Zielregister nicht angegeben.)

```
lw 0xDC
lw 0x90
lw 0xD3
lw 0x0C
lw 0xDD
lw 0x93
lw 0xD0
lw 0x0D
```

- a) Geben Sie zu jedem Lesebefehl an, ob es sich um einen Hit oder Miss handelt. Berechnen Sie anschließend die Hitrate.
- b) Geben Sie die abschließende Belegung des Caches an.
- c) Wie verändert sich die Hit-Rate, wenn wir den direkt abbildenden Cache zu einen zweifach assoziativen Cache umwandeln? Der Aufbau der Adresse bleibt identisch. Wie verändert sich die Größe des Caches unter dieser Annahme?
- d) Welche Möglichkeit gibt es grundsätzlich, um die Anzahl an Cache-Misses zu reduzieren? Auf welchem Prinzip basiert diese?

Aufgabe 3 Cache

In der Moodle-Aufgabe finden Sie ein Programm `array_copy.s`. Importieren Sie dieses Programm in Ripes und führen Sie es aus. Beobachten Sie die Ausführung auch aus der Cache-Ansicht in Ripes. Machen Sie sich mit den dortigen Optionen zur Konfiguration des Datencaches vertraut.

1. Geben Sie drei mögliche Konfigurationen für einen Cache der Größe 32 Words an. Nutzen Sie dabei stets die Strategien *Write-back* und *Write allocate* und beschreiben Sie knapp das Verhalten, das durch diese Strategien erzielt wird.
2. Manchmal ist *Write allocate* nicht sinnvoll; die Alternative wird manchmal *write around* genannt. Beschreiben Sie entsprechende Situationen. Wie könnte eine ISA ergänzt werden, um einen Assembler-Programme die Kontrolle zwischen *write allocate* und *write around* zu lassen? Wie schlägt sich das in C-Code nieder?

Verwenden Sie nun in Ripes einen direkt abbildenden Datencache mit 2^3 Lines und 2^2 Words/Line. Die folgenden Aufgaben beziehen sich ausschließlich auf diesen Datencache und nicht auf den Instruktionscache.

3. Führen Sie das Programm mit folgenden Strategie-Konfigurationen jeweils mit einem `offset` von 64 sowie einem `offset` von 128 aus:
 - *Write-back, Write allocate*
 - *Write-through, Write allocate*
 - *Write-back, No write allocate*
 - *Write-through, No write allocate*

Der `offset`-Wert kann im `.data`-Segment angepasst werden. Vergleichen Sie die Strategien mithilfe einer geeigneten Metrik.

4. Erklären Sie ihre Beobachtungen aus der vorherigen Teilaufgabe. Welches besondere Problem entsteht bei der Strategie *Write allocate* in Verbindung mit einem `offset` von 128? Mit welcher Cache-Konfiguration kann dieses Problem vermieden werden ohne die Gesamtgröße des Caches (32 Words) zu ändern?
5. Leider liefert ihr Cache-Produzent nur die oben vorgegebene Cache-Konfiguration. Modifizieren Sie deshalb stattdessen das Programm, um die Hit-Rate für die gegebenen Strategie-Kombinationen und `offset`-Werte zu maximieren.

Aufgabe 4 Cache Kohärenz

Betrachten Sie im Folgenden einen 4-Kern-Prozessor dessen Cache wie folgt aktualisiert wird. Jeder der Kerne ist in einem Ring miteinander verbunden. Wird in einem Takt der Cache eines Kerns geändert, so werden diese Änderungen an die Caches aller anliegenden Kerne übertragen. Pro durchlaufenen Kern wird hierbei ein Takt benötigt. Erhält ein Kern nun eine Änderung, so gibt er diese rechtsbündig (siehe Abbildung) weiter. Auf diese Weise werden schließlich alle Kerne

aktualisiert. Beim Schreiben und Aktualisieren wird dabei immer der aktuellere Wert übernommen. Die folgende Abbildung enthält einen beispielhaften Ablauf bei einer Änderung von Kern 0.

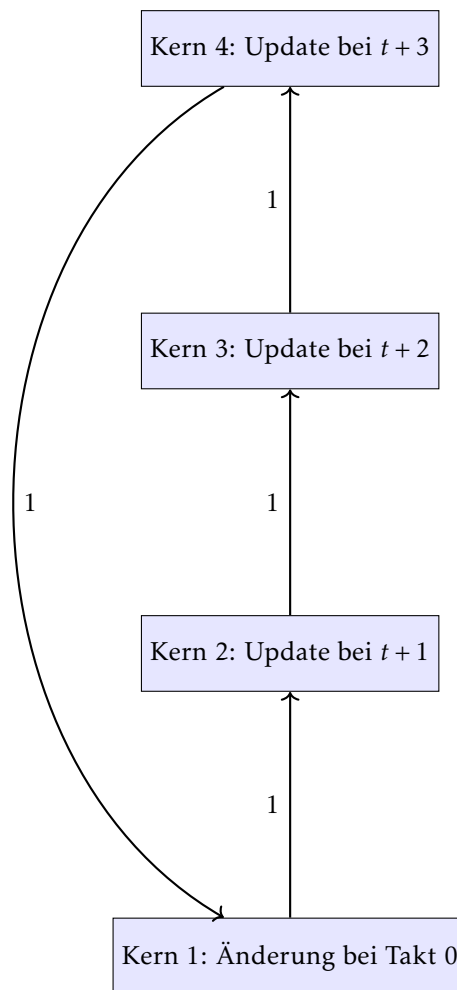


Abbildung 1: Beispiel des Aktualisierungsverhalten

Hält dieses Prozessor-/Cache-Modell die sequenziellen Kohärenz-Regeln? Falls nicht, was müsste man an diesem Modell ändern so dass diese Regeln eingehalten werden? Wie würde man dies umsetzen?